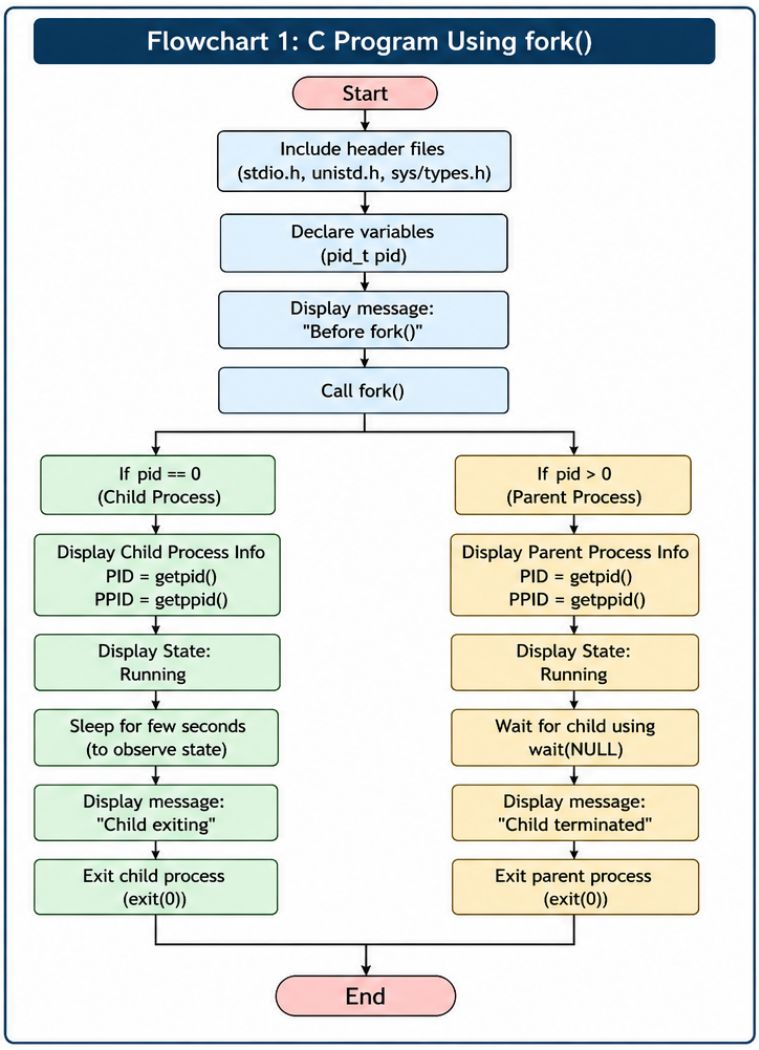
Task 3: Develop a C program using fork() that creates a parent and child process. Display the Process ID (PID), Parent Process ID (PPID), and process states at different stages of execution.

Flow Chart




```
#include <stdio.h>

#include <stdlib.h>

#include <unistd.h>

#include <sys/types.h>

#include <sys/wait.h>


int main() {
```

```
pid_t pid;

printf("Before fork():\n");
printf("Process PID = %d, PPID = %d\n", getpid(), getppid());

pid = fork();

if (pid < 0) {
    // Fork failed
    fprintf(stderr, "Fork failed\n");
    exit(1);
}
else if (pid == 0) {
    // Child process
    printf("\n[CHILD] Process created.\n");
    printf("[CHILD] PID = %d, PPID = %d\n", getpid(), getppid());
    printf("[CHILD] State: Running\n");
    sleep(2);   // Simulate work
    printf("[CHILD] Terminating.\n");
    exit(0);
}
else {
    // Parent process
    printf("\n[PARENT] Child created with PID = %d\n", pid);
    printf("[PARENT] PID = %d, PPID = %d\n", getpid(), getppid());
    printf("[PARENT] State: Waiting for child to terminate\n");
```

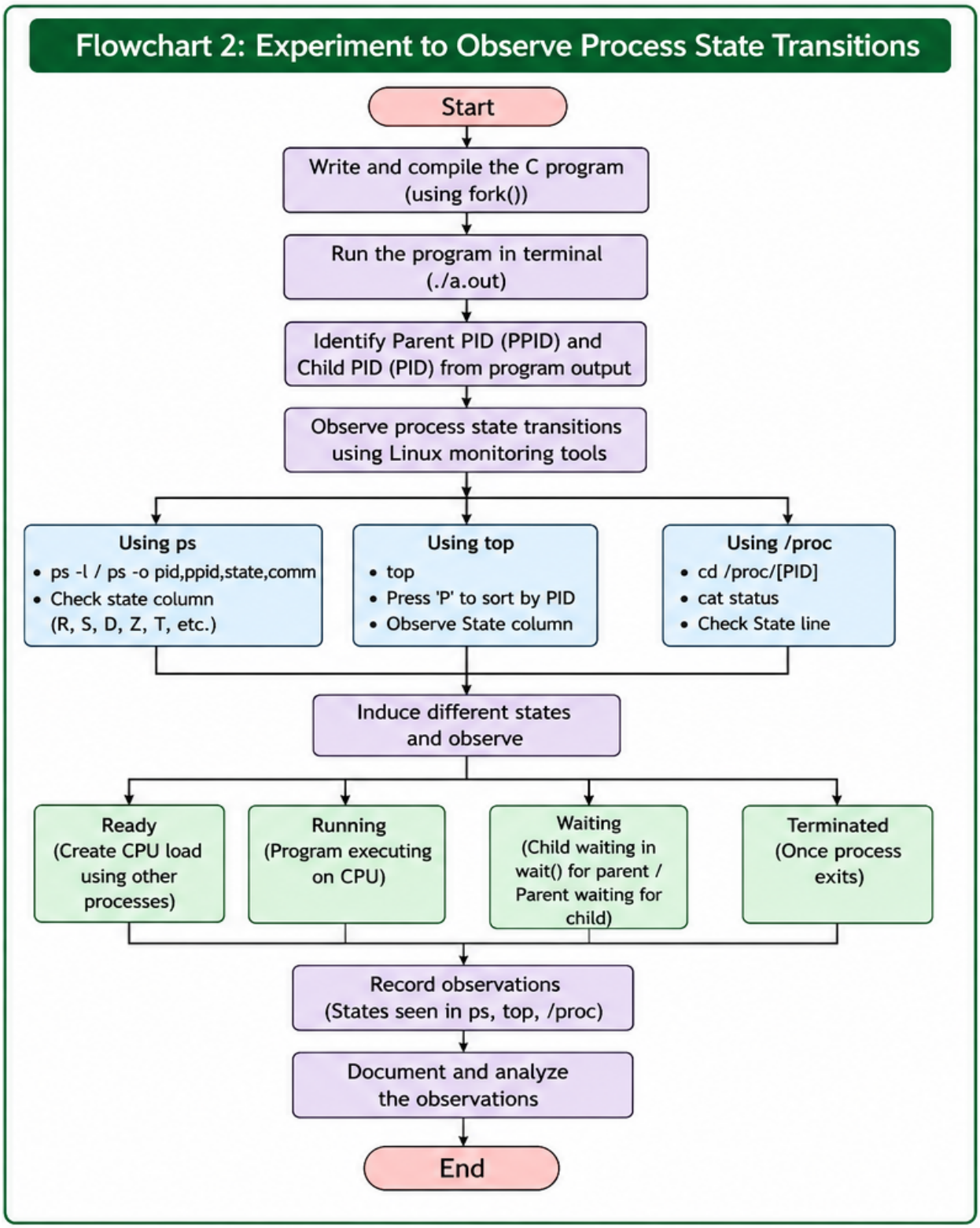
```
        int status;
        waitpid(pid, &status, 0);   // Parent waits (Waiting state)

        printf("[PARENT] Child (PID = %d) has terminated.\n", pid);
        printf("[PARENT] State: Running -> Terminated\n");
    }

    return 0;
}
```

Execution:

```
Parent Process Started
Parent PID: 21749
Child 1 started. PID = 21753, Parent PID = 21749
Child 2 started. PID = 21754, Parent PID = 21749
All 3 child processes have been created.

Using wait():
Child 3 started. PID = 21755, Parent PID = 21749
Child completed. PID = 21753
wait() detected child PID 21753 finished with exit status 10

Using waitpid():
waitpid() detected child PID 21754 finished with exit status 11
waitpid() detected child PID 21755 finished with exit status 12

Parent process completed.
```

Task 4: Design an experiment to observe process state transitions (Ready, Running, Waiting, Terminated) using Linux monitoring tools such as ps, top, and /proc. Document the observations.

Flow Chart:


Flowchart 2: Experiment to Observe Process State Transitions
Start
Write and compile the C program (using fork())
Run the program in terminal (./a.out)
Identify Parent PID (PPID) and Child PID (PID) from program output
Observe process state transitions using Linux monitoring tools
Using ps
ps -l / ps -o pid,ppid,state,comm
Check state column (R, S, D, Z, T, etc.)
Using top
top
Press 'P' to sort by PID
Observe State column
Using /proc
cd /proc/[PID]
cat status
Check State line
Induce different states and observe
Ready (Create CPU load using other processes)
Running (Program executing on CPU)
Waiting (Child waiting in wait() for parent / Parent waiting for child)
Terminated (Once process exits)
Record observations (States seen in ps, top, /proc)
Document and analyze the observations
End

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main() {
   printf("PID = %d : Running (CPU-bound work)\n", getpid());
   for (long i = 0; i < 2000000000; i++);   // busy-loop -> Running

   printf("PID = %d : Going to sleep -> Waiting/Blocked state\n", getpid());
   sleep(15);   // sleep() -> Waiting (Interruptible sleep, state 'S')

   printf("PID = %d : Woke up, finishing -> Terminated\n", getpid());
   return 0;
}
```

Execution: